

Problem Statement

To implement a Client-Server messaging demonstrating **Hamming Code** using sockets in Python

Procedure

Server-Side Implementation (hserver.py)

- Accept user input and convert it to binary format.
- Calculate the necessary number of parity bits using the Hamming code algorithm.
- Integrate the parity bits into the binary data to form the Hamming codeword.
- Set up a socket server that listens for incoming connections.
- Send the Hamming encoded message to the connected client.

Client-Side Implementation (client.py)

- Set up a socket client and connect to the server.
- Receive the Hamming encoded message from the server.
- Decode the message by checking the parity bits to detect any errors.
- Identify and correct any single-bit errors in the received message.
- Extract the original data from the corrected message.
- Introduce a controlled error by inverting a specific bit in the received message.
- Re-evaluate the message with the introduced error to demonstrate error detection and correction capabilities.

Output

```
20BCE2356 | Shahil Mohammed
→ hamming python3 hserver.py
-- 20BCE2356 -- | Hamming Code | Sender

Enter text: cat
Binary of the input text: 110001111000011110100
Parity Bits: 5
Hamming Codeword: 01101001011110010011110100

Server listening on localhost:2356
Connection from ('127.0.0.1', 62291)
→ hamming
```

```
20BCE2356 | Shahil Mohammed
→ hamming python3 client.py
-- 20BCE2356 -- | Hamming Code | Receiver

Enter port number: 2356
Received: 01101001011110010011110100
No Error in the received message!
Hamming Decoded Message: 110001111000011110100

- Introducing error by inverting the third bit of the received message -
Bit Inverted Message for testing: 01001001011110010011110100

- Error Testing Results-
Error detected at position: 3
Hamming Decoded Message with Error: 010001111000011110100
→ hamming
```

Problem Statement

To implement a Client-Server messaging demonstrating **Cyclic Redundancy Code** using sockets in Python

Procedure

Server-Side Implementation (cserver.py)

- Accept user input and convert it to binary format.
- Append CRC checksum to the binary data using the given CRC polynomial key.
- Set up a socket server that listens for incoming connections.
- Send the CRC encoded message to the connected client.

Client-Side Implementation (client.py)

- Set up a socket client and connect to the server.
- Receive the CRC encoded message from the server.
- Verify the received message by recalculating the CRC checksum.
- Display whether the received message is error-free or contains errors.
- Introduce a controlled error by modifying a bit in the received message.
- Re-evaluate the message with the introduced error to demonstrate error detection capabilities.

Problem Statement

To implement a Client-Server messaging demonstrating **Checksum** using sockets in Python

Procedure

Server-Side Implementation (checksum.py)

- Accept user input and convert it to binary format.
- Calculate the checksum for the binary data.
- Append the checksum to the original binary data to form the codeword.
- Set up a socket server that listens for incoming connections.
- Send the checksum encoded message to the connected client.

Client-Side Implementation (client.py)

- Set up a socket client and connect to the server.
- Receive the checksum encoded message from the server.
- Verify the received message by recalculating the checksum.
- Display whether the received message is error-free or contains errors.

```
# Using join() + ord() + format()
```

```
res = ''.join(format(ord(i), '08b') for i in test_str)
```